EJERCICIO 1

Calcula la complejidad en notación O grande de todas las operaciones de la lista enlazada simple vista en este tema (implementada con dos punteros `first` y `last`).

Operaciones básicas	Complejidad
procedure initialize(var list: tlista);	O(1)
function is_empty(list: tlista): boolean;	O(1)
function first(list: tlista): integer;	O(1)
function last(list: tlista): integer;	O(1)
procedure insert_at_end(var list: tlista; x: integer);	O(1)
procedure insert_at_begin(var list: tlista; x: integer);	O(1)
procedure delete_at_end(var list: tlista);	O(n)
procedure delete_at_begin(var list: tlista);	O(1)
function in_list(list: tlista; x:integer): boolean;	O(n)
function rec_in_list(list:tlista; x:integer): boolean;	O(n)
Operaciones adicionales	**Complejidad**
procedure show(list: tlista);	O(n)
procedure clear(var list: tlista);	O(n)
function num_elems(list: tlista): integer;	O(n)
procedure copy(list: tlista; var c2: tlista);	O(n)

EJERCICIO 2

Implementa el TAD Deque genérico en función de un elemento genérico TElemento.

```
unit uDeque;

uses uTElemento;

interface

  type
    nodo = record
      info: TElemento;
      sig: ^nodo;
    end;

    tDeque = record
      first, last: ^nodo
    end;

  { Operaciones básicas }
  procedure initialize(var d: tDeque);
  procedure enqueue(var d: tDeque; x: TElemento);
  procedure insert_first(var d: tDeque; x: TElemento);
  procedure dequeue(var d: tDeque);
  function first(d: tDeque): TElemento;

  { Otras operaciones }
  function last(d: tDeque): TElemento;
  function num_elems(var d: tDeque): integer;
  function is_empty(d: tDeque): boolean;

implementation

  procedure initialize(var d: tDeque);
  begin
    d.first := nil;
    d.last := nil;
  end;

  procedure enqueue(var d: tDeque; x: TElemento);
  var
```

```
    new_node: ^nodo;
  begin
    new(new_node);
    assign(new_node^.info, x);
    new_node^.sig:= nil;
    if is_empty(d) then
      d.first:= new_node
    else
      d.last^.sig:= new_node;
    d.last:= new_node;
  end;

  procedure insert_first(var d: tDeque; x: TElemento);
  var
    new_node: ^nodo;
  begin
    new(new_node);
    assign(new_node^.info, x);
    new_node^.sig:= d.first;
    if is_empty(d) then d.last:= new_node;
    d.first:= new_node;
  end;

  procedure dequeue(var d: tDeque);
  var
    aux: ^nodo;
  begin
    if not is_empty(d) then
    begin
      aux:= d.first;
      d.first := d.first^.sig;
      if d.first = nil then d.last:= nil;
      dispose(aux);
    end;
  end;

  function first(d: tDeque): TElemento;
  { PRE: deque not empty }
  begin
    assign(first,d.first^.info);
  end;

  function last(d: tDeque): TElemento;
  { PRE: deque not empty }
  begin
    assign(last,d.last^.info);
  end;

  function num_elems(d: tdeque): integer;
  var
    aux: ^nodo;
    count: integer;
  begin
    count:= 0;
    aux:= d.first;
    while aux<>nil do
    begin
        count:= count + 1;
        aux:= aux^.sig
    end;
    num_elems:= count
  end;

  function is_empty(d: tDeque): boolean;
  begin
    is_empty:= (d.first = nil) and (d.last = nil);
  end;

end.
```

EJERCICIO 3a

Implementa una operación que permita invertir una lista enlazada simple.

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(1)

```
(1) Versión iterativa

  procedure reverse(var list: tlista);
  var
   anterior, actual, aux: ^nodo;
  begin
   anterior := nil;
   actual := list.first;
   while (actual<>nil) do
   begin
      aux := actual^.sig;
      actual^.sig:= anterior;
      anterior := actual;
      actual: = aux;
   end;
   list.last:= list.first;
   list.first:= anterior;
  end;

(2) Versión recursiva I

  procedure rec_reverse(var list:tlista);
  var
    resto: tlista;
  begin
    if (not is_empty(list)) and (list.first<>list.last) then
      begin
        // sepàramos el primero del resto
        resto.first:= list.first^.sig;
        if list.first^.sig = nil then resto.last:=nil
        else resto.last:= list.last;
        // damos la vuelta recursivamente al resto
        rec_reverse(resto);
        // añadimos el primero al final de  la lista resto invertido
        insert_at_end(resto,list.first^.info);
        // eliminamos el primer nodo de la lista original que no usaremos más
        dispose(list.first);
        // la nueva lista original es el resto invertido con el primero al final
        list.last:= resto.last;
        list.first:=resto.first;
      end;
  end;

(3) Versión recursiva II

  procedure rrec_reverse(var list:tlista);
  { Otra versión recursiva de "reverse" }
  var
    resto: tlista;
  begin
    if (not is_empty(list)) and (list.first<>list.last) then
      begin
        // sepàramos el primero del resto
        resto.first:= list.first^.sig;
        if list.first^.sig = nil then resto.last:=nil
        else resto.last:= list.last;
        // damos la vuelta recursivamente al resto
```

```
      rec_reverse(resto);
      // añadimos el primero al final de la lista resto invertido
      resto.last^.sig:=list.first;
      list.first^.sig:=nil;
      // la nueva lista original es el resto invertido con el primero al final
      list.last:= list.first;
      list.first:=resto.first;
    end;
end;
```

EJERCICIO 3b

Implementa una operación que permita invertir una lista enlazada doble.

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(1)

```
procedure reverse(var dlist: tlistaDoble);
var
 anterior, actual, aux: ^nodo;
begin
 anterior := nil;
 actual := dlist.first;
 while (actual<>nil) do
 begin
    aux := actual^.sig;
    actual^.sig:= anterior;
    actual^.prev:= aux;
    anterior := actual;
    actual: = aux;
 end;
 dlist.last:= dlist.first;
 dlist.first:= anterior;
end;
```

Nota: La implementación recursiva no es complicada y su enfoque de resolución es similar a la de las listas simples, por lo que se propone como ejercicio avanzado.

EJERCICIO 4

Implementa mediante una lista ordenada las operaciones de inserción y servicio (pop) de una cola de procesos con prioridad (0-máxima prioridad, 3-minima).

Complejidad de tiempo esperada inserción: O(n)
Complejidad de tiempo esperada servicio (pop): O(n)
Espacio auxiliar esperado: O(1)

```
procedure insort(var cola: tcolaProcesos; p: tProceso);
{ Inserción ordenada en cola de procesos con prioridad (0-máxima, 3-minima) }
var
  new_node, actual: ^nodo;
begin
  new(new_node);
  new_node^.info:= p;
  if (is_empty(cola)) or (p.prioridad < cola.first^.info.prioridad) then
    begin // insertar el primero
      new_node^.next:= cola.first;
      cola.first:= new_node;
    end
  else
    begin  // Situar actual en el nodo anterior a la inserción
```

```
        actual:= cola.first;
        while (actual^.next <> nil) and
              (p.prioridad >= actual^.next^.info.prioridad) do
            actual:= actual^.next;
        new_node^.next:= actual^.next;
        actual^.next:= new_node;
      end;
  end;
```

EJERCICIO 4bis

Implementa el sistema anterior mediante una cola multinivel de 4 colas.

Complejidad de tiempo esperada inserción: O(n)
Complejidad de tiempo esperada servicio (pop): O(n)
Espacio auxiliar esperado: O(1)

```
program cola_prioridades_multinivel;
uses
  uColaProcesos, crt;

const MIN_PRIORIDAD = 3;
      MAX_PROCESOS = 10;

type
  tprioridad = 0..MIN_PRIORIDAD;
  tcola_procesos_multinivel = array[tprioridad] of tcolaProcesos; // en uColaProcesos

var
  sistema_procesos: tcola_procesos_multinivel;
  proceso_prueba: tProceso;
  pproceso: ^tProceso;
  i: integer;

procedure inicializar_sistema(var sist: tcola_procesos_multinivel);
var
  i:integer;
begin
   for i:=0 to 3 do initialize(sist[i])
end;

procedure introducir_proceso(var sist: tcola_procesos_multinivel; p: tProceso);
begin
   enqueue(sist[p.prioridad], p);
end;

procedure servir_proceso(var sist: tcola_procesos_multinivel);
var
  encontrado_proceso: boolean;
  prioridad_servida: integer;
  proceso_a_servir: tProceso;
begin
   encontrado_proceso:= false;
   prioridad_servida:= 0;
   while (not encontrado_proceso) and (prioridad_servida <= MIN_PRIORIDAD) do
      if not is_empty(sist[prioridad_servida]) then
         encontrado_proceso:= true
      else
         prioridad_servida:= prioridad_servida+1;
   if encontrado_proceso then
     begin
        proceso_a_servir := first(sist[prioridad_servida]);
        writeln('Sirviendo proceso en cola: ', prioridad_servida);
```

```
        mostrar_proceso(proceso_a_servir);
        dequeue(sist[prioridad_servida]);
      end
   else writeln('No hay procesos en espera en el sistema');

end;

begin  // Programa de prueba para probar los procedimientos anteriores
   inicializar_sistema(sistema_procesos);
   randomize;
   for i:=1 to MAX_PROCESOS do
    begin
      new(pproceso);
      str(random(1000),pproceso^.id);
      pproceso^.id:= 'P'+pproceso^.id;
      pproceso^.prioridad:=random(4);
      introducir_proceso(sistema_procesos, pproceso^);
    end;
   for i:=1 to MAX_PROCESOS do servir_proceso(sistema_procesos);
   repeat until keypressed
end.
```

EJERCICIO 5

Implementa la operación de eliminación de un elemento por clave en una lista ordenada

Complejidad de tiempo esperada: O(n)
Espacio auxiliar esperado: O(1)

```
(1) Versión iterativa:

  procedure delete_value(var list: tLista; x: integer);
  var
    actual, anterior: ^nodo;
    encontrado: boolean;
  begin
    if not is_empty(list) then
      if (list.first^.info = x) then // El valor a eliminar es el primero de la lista
        begin
          if (list.first = list.last) then list.last:= nil;
          actual:= list.first;
          list.first:= list.first^.sig;
          dispose(actual);
        end
      else
        begin // Solo elimina la primera aparición del valor a eliminar
          encontrado:= false;
          anterior:= list.first;
          actual:= anterior^.sig;
          while (actual <> nil) and (actual^.info <= x) and (not encontrado) do
            begin
              if actual^.info = x then
                begin
                   anterior^.sig:= actual^.sig;
                   if actual = list.last then list.last:=anterior;
                   dispose(actual);
                   encontrado:= true
                end;
                anterior:= actual;
                actual:= actual^.sig;
            end;
        end;
  end;
```

Nota: *lo único que cambia con respecto a una lista normal es "and (x<=actual^.info)" en el while, asumiendo que se trata de una lista ordenada ascendentemente. La condición extra comprueba que el elemento esté en el lugar que le corresponde, si no para el bucle dado que más adelante ya no estará pues la lista está ordenada.*

(2) Versión recursiva:

```
procedure delete_value_rec(var list: tlista; x: integer);
var
  aux:^nodo;
  resto: tlista;
begin
  if not is_empty(list) and (x <= list.first^.info) then
    begin
      if (list.first^.info = x) then
        begin // hay que eliminar el primero
          aux:= list.first;
          list.first:= list.first^.sig;
          if (list.first = nil) then list.last:= nil;
          dispose(aux);
        end
      else
        begin
           // Preparar la lista resto
           resto.first:= list.first^.sig;
           if resto.first = nil then resto.last:=nil
           else resto.last:= list.last;
           // Llamada recursiva
           delete_value(resto,x);
           // Recolocar la lista original
           list.first^.sig:=resto.first;
           list.last:=resto.last
        end;
    end;
end;
```

Nota: de nuevo, la única diferencia con la implementación para una *"lista normal" es "and (x<=list.first^.info)" en el if, asumiendo que se trata de*

EJERCICIO 6 (reto)

A partir de un archivo de texto con tweets, genera un diccionario en el que la clave serán los hashtags y los valores serán los tweets que contengan dicho hashtag. Utiliza una lista ordenada para la lista de claves y calcula la complejidad de la operación de inserción.